



BUIDL CTC 2026 FALL · TRACK AI

ProveOut

Money moves when the work is proved, and anyone can prove it failed.

Job escrow on Creditcoin CC3 · settled by Attestcoin readability · no
oracle operator, no arbiter

An AI agent working across chains cannot be paid safely.

Pay first and the buyer loses the money if the work never happens. Pay after and the agent has no guarantee it will ever be paid.

Every existing answer inserts a third party to decide whether the work was done: an oracle operator, an arbiter, or the platform itself. Someone, somewhere, presses a button.

Evidence that only the winner submits is not evidence.

The standard shape of a cross-chain settlement system is that the party who benefits watches for their own good news and submits it. A worker sees the repayment and posts it. A builder finishes the job and posts the receipt.

Nobody posts their own failure.

Such a system does not need to censor bad news. Bad news simply never arrives, because the only party watching has no reason to send it, and the escrow quietly waits out its timeout.

ProveOut pays strangers to submit it.

Verification and payout in one Creditcoin transaction.

ETHEREUM SEPOLIA · SOURCE

```
WorkOracle.sol  reporter-gated
├ WorkCompleted(jobId, criteriaHash,
|               builder, outputHash)
└ WorkFailed(jobId, reason)
```



CREDITCOIN CC3 · SETTLEMENT

```
SourceRegistry.sol
└ emitter → chainKey, evmChainId,
  topic0s, field positions

JobEscrow.sol is ASCBase
├ execute(0) → release
├ execute(1) → challenge, any caller
└ refund() after the deadline
```

A relayer waits for attestation, fetches the proof, and submits it. It is untrusted and anyone can be it: if it lies the contract rejects it, if it disappears anyone else can submit the same proof.

The protocol's own entry point, not a wrapper around it.

JobEscrow inherits **ASCBBase** and settles through the protocol's `execute()`. Replay protection and the query id are Attestcoin's, derived by the precompile from the verified Merkle path, not a caller-supplied transaction hash.

Block Prover precompile `0x0000000000000000000000000000000000000000000000000000000000000000FD2`, called with a continuity proof and a Merkle proof from the Proof Builder service. We build neither.

`receiptStatus == 1` is mandatory. The precompile proves inclusion, not success.

EvmV1Decoder is imported from the published package, never copied in. A copy keeps compiling and starts decoding garbage the first time a field moves.

Two kinds of source, configurable. An attesting oracle whose event names the job, or **any ordinary ERC-20**, where the acceptance criterion is the Transfer itself. The second kind has no source-side trust assumption at all. Adding either is one `registerSource` call, not a code change.

Inclusion is not success.

```
EvmV1Decoder.ReceiptFields memory receipt = ...;  
if (receipt.receiptStatus != 1)  
    revert ReceiptNotSuccessful(receipt.receiptStatus);
```

The precompile proves a transaction was included in a block. **A reverted transaction is in the block too.** Without this line a builder sends a completion call, lets it revert, proves that inclusion completely honestly, and is paid for work that never happened. Every proof in that attack is genuine.

This is the pattern Gluwa's own reference contract uses. We did not invent it. We refused to skip it.

SIX GATES ON THE SETTLEMENT PATH

GATE	CHECK	WHAT IT STOPS
G1	<code>receiptStatus == 1</code>	a reverted call being paid as a success
G2	<code>emitter</code> is registered	anyone deploying their own oracle and draining every escrow
G2b	proved chain id matches	the same bytecode redeployed on another attested chain
G3	exact <code>topic0</code> and <code>topic count</code>	a different event settling a job
G4	fields read from stored positions	indexed parameters read out of data as plausible garbage
G5	<code>query id</code> unseen	the same proof settling twice
G6	within the deadline	release and refund racing on transaction ordering

The challenge path reuses G1 to G5 and deliberately skips G6. A failure stays provable forever, or hiding one until the clock ran out would be a winning strategy.

ON CHAIN, NOT ON A SLIDE

Five real transactions on CC3 Testnet.

01 RELEASED A proved **WorkCompleted** paid the builder **1,200 tUSDC**

0x02c42a8e6f780daef8ca59f985e2c2d008be2625df61d0b775a2a51b307ca23a

02 REFUNDED BY A STRANGER

A proved **WorkFailed** carried by **0xA0356B8011B63990978f2a7CCc389c3769d092Ea**, neither buyer nor builder. Buyer refunded **1,100**, that wallet paid a **100 tUSDC** bounty from the bond

0x90b4e9f403712c3d29c45d0315ef7ce1f4e0a47c8d2686dd9e38397aa4a471e8

03 NO ORACLE AT ALL

Settled by **canonical WETH on Sepolia**, a token with no stake in the job. The criterion was an on-chain delivery of at least 0.001 WETH to the buyer. Nobody can emit that Transfer without moving the tokens

0x21c775e8943d9691e5d50cc5b179e42fcdd6cf5218d5ffb3f8fcfb9026b522a4

04 REVERTED, ON PURPOSE Proof 01 replayed. **Query already processed**

0xf880148cf34fdc17fd3eb59ec7fbd26e09f066f89088367cdc4e43601908e96e

05 REFUSED AT THE SOURCE The builder tried to certify their own job on Sepolia. **NotReporter**, before any proof could exist

0x57d5187175be6066ed63e0e3c6d3580cd6437c7ab623b257392e571ce332436f

The cost of being caught falls on whoever failed.

PARAMETER	VALUE
Job payout	1,000 tUSDC
Builder bond	20% of the payout · 200 tUSDC
Challenger bounty	50% of the bond · 100 tUSDC
On release, builder receives	1,200 tUSDC payout + bond returned
On challenge, buyer receives	1,100 tUSDC
On challenge, challenger receives	100 tUSDC out of the builder's bond

The vault rule is $\text{balance} + \text{paidOut} \geq \text{takenIn}$, an inequality rather than an equality, so a stranger sending one micro-unit into the vault cannot wedge every escrow in it.

CHECK IT WITHOUT TRUSTING US

Anyone can verify a proof, with no wallet and nothing spent.

The precompile's `verify` is a view function, so the real verification path can be exercised against real attestation state for free.

IN THE BROWSER

`proveout.vercel.app/verify`

Paste any Sepolia transaction hash. It fetches a real Merkle and continuity proof, calls the precompile on Creditcoin, and shows the result with every log recovered from the proved bytes.

```
test_release_isRefused_whenReceiptStatusIsZero ·
test_anyone_canForceRefund_withFailureProof ·
test_builder_cannotCertifyTheirOwnWork ·
test_releaseAndRefundWindows_neverOverlap
```

IN THE REPO

`spikes/spike1_prove.ts`

The same call from a terminal, unfunded. It is how the project's first gate was cleared before a line of contract code was written.

TESTS

118 passing, 0 failing

Including a 6,144-call invariant run on the vault balance rule, and a mocked precompile so the suite runs offline.

What this does NOT do.

Attestcoin proves inclusion, not exclusion. It can prove a transaction happened. It cannot prove none happened. ProveOut does not make hiding a failure impossible; it makes it detectable and expensive. That is the whole promise of the challenge mechanism, and not a word more.

An attested job trusts its oracle within its own scope. Whoever holds a reporter role decides what is provable there. A delivery job does not have this assumption, because the token is not ours and cannot be asked to lie. Choosing between them is a product decision.

TestUSDC is not USDC. A six decimal demo token with an open mint and no value.

Testnet only, and unaudited. Nothing here has had a security review.

The bond ratio is a demo parameter, not a modelled economic result.

Only transaction types 0 and 2 are accepted. Those are the two the shipped decoder fully decodes; anything else is rejected loudly rather than decoded on a guess.

The relayer is not trusted, but it is not yet redundant. Anyone can submit proofs; today only our worker does it automatically.



BUIDL CTC 2026 FALL · TRACK AI

Money moves when the work is proved, and anyone can prove it failed.

Live	<code>proveout.vercel.app</code>
Verify a proof	<code>proveout.vercel.app/verify</code>
Repository	<code>github.com/yukitran03/proveout</code>
JobEscrow	<code>0xC69D0f7a0f4A59db88b74ef64439B643b8c9B65b</code>
WorkOracle	<code>0xF38ac85b0cEC258dF08e8a45446f4f156EC591e9</code>
Chain	Creditcoin CC3 Testnet, id 102031 · source Sepolia, chain key 1
Explorer	<code>creditcoin-testnet.blockscout.com</code>